ERHVERVSAKADEMI
AARHUS



Programmering modul 02

CSIK E2026

ERHVERVSAKADEMI
AARHUS

I dag

Syntaks og semantik af udtryk igen

Relationelle og logiske udtryk

Betingede udtryk

Kortslutninger og sideeffekter

5+20+5 minutters pause undervejs

Slutter 16:00

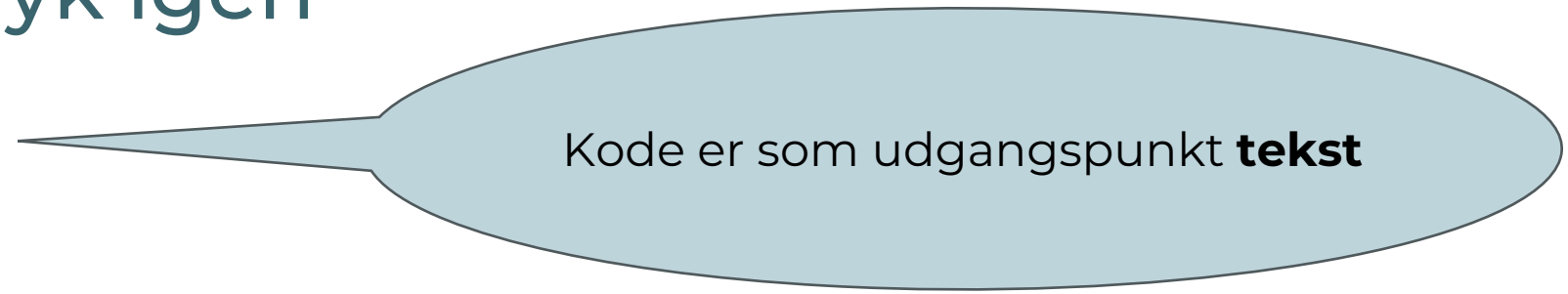
Syntaks for udtryk igen

```
(7 + x) * (8 - y / 3)
```

ERHVERVSAKADEMI
AARHUS

Syntaks for udtryk igen

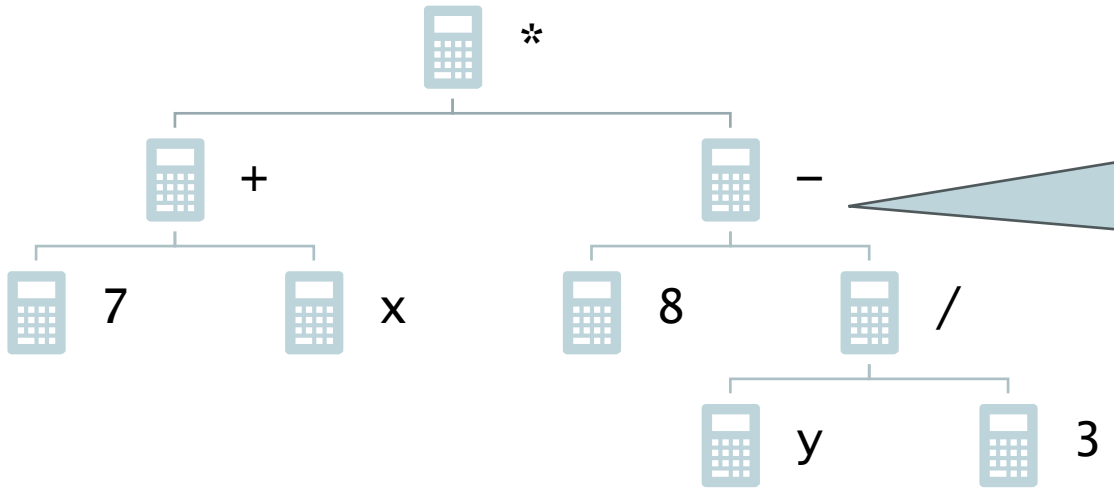
```
(7 + x) * (8 - y / 3)
```



Syntaks for udtryk igen

```
(7 + x) * (8 - y / 3)
```

Kode er som udgangspunkt **tekst**

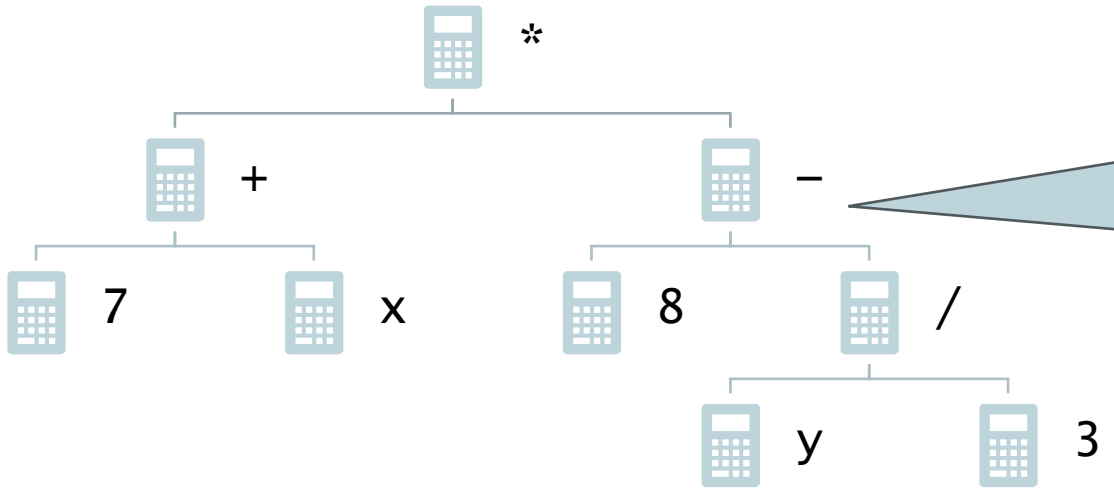


Programmeringssprogets syntaks-regler fortæller, hvordan teksten skal opfattes som et **træ**

Syntaks for udtryk igen

```
(7 + x) * (8 - y / 3)
```

Kode er som udgangspunkt **tekst**

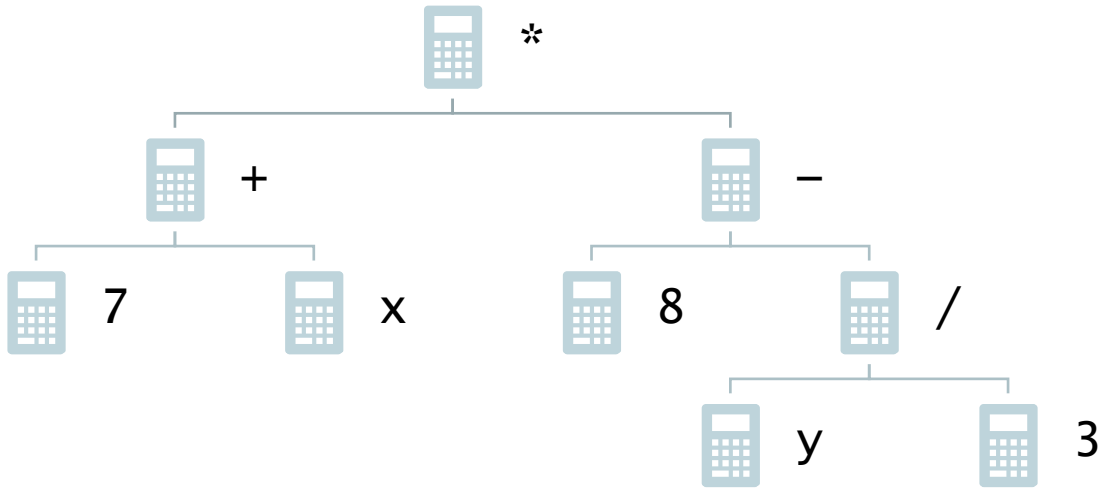


Programmeringssprogets syntaks-regler fortæller, hvordan teksten skal opfattes som et **træ**

Processen, der tager os fra tekst til træ, kaldes **syntaksanalyse** eller **parsing**

Semantik af udtryk

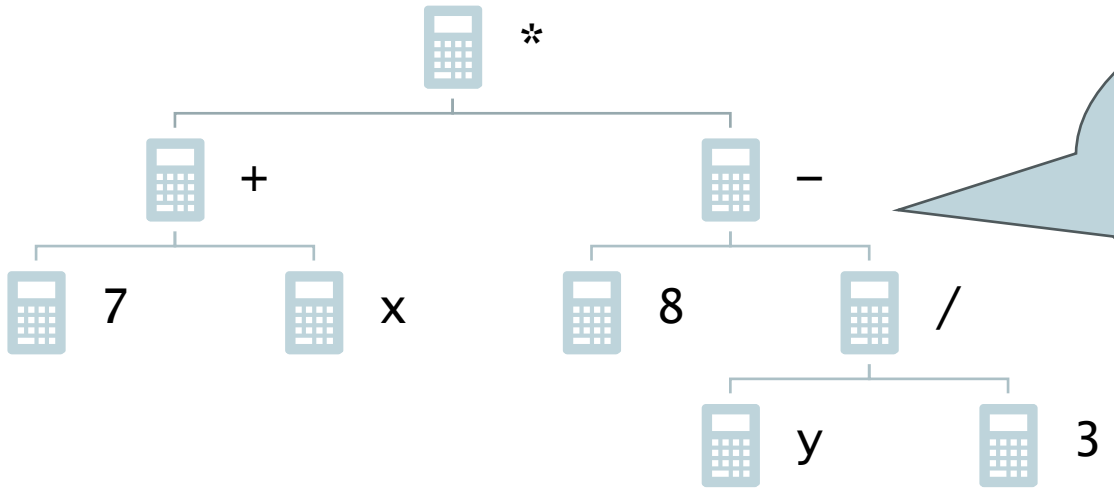
```
(7 + x) * (8 - y / 3)
```



ERHVERVSAKADEMI
AARHUS

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```



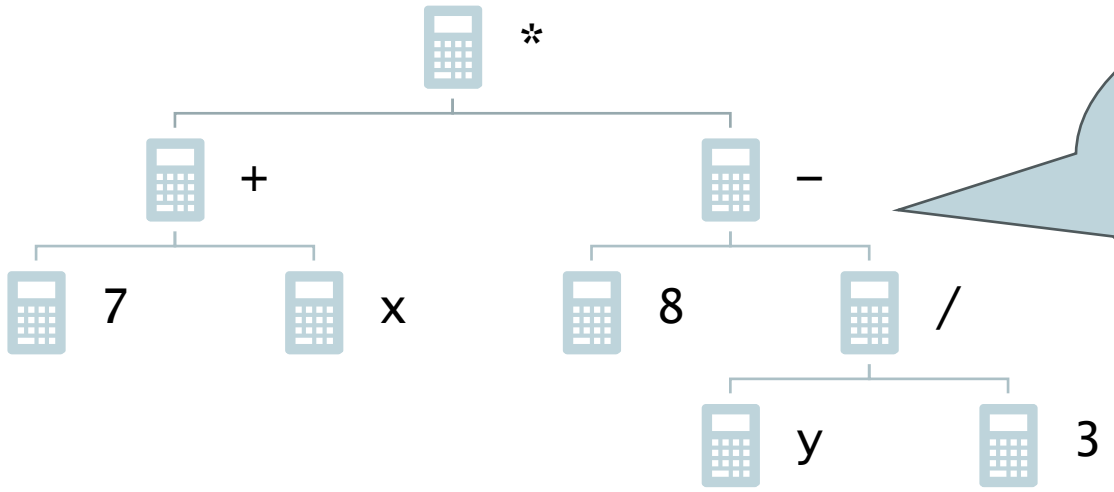
Programmeringssprogets semantiske regler fortæller, hvordan syntakstræet bruges til at beregne den værdi, udtrykket beskriver

Processen, der tager os fra syntakstræ til værdi kaldes **evaluering**

ERHVERVSAKADEMI
AARHUS

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

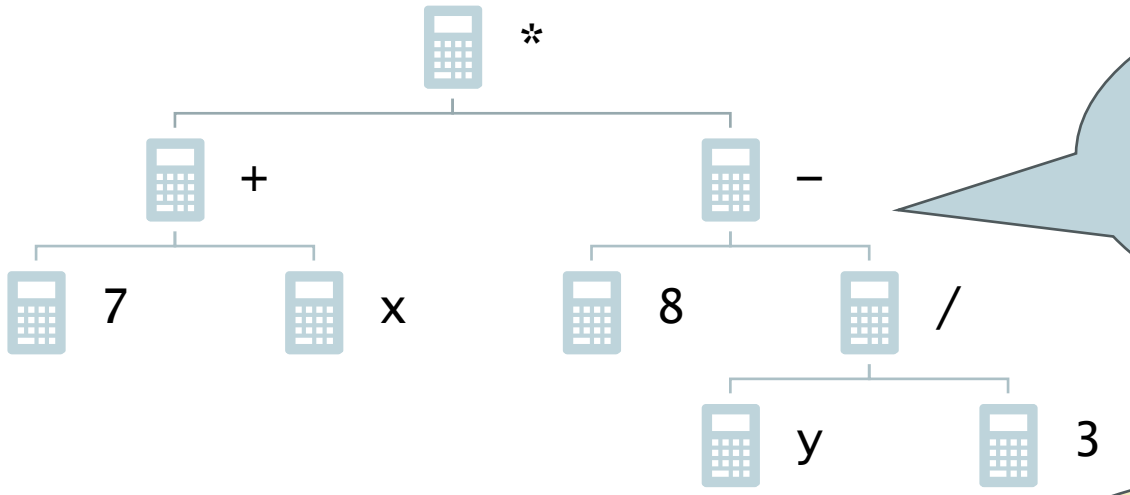


Programmeringssprogets semantiske regler fortæller, hvordan syntakstræet bruges til at beregne den værdi, udtrykket beskriver

Evaluering kan ske på en simpel **stakmaskine**

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

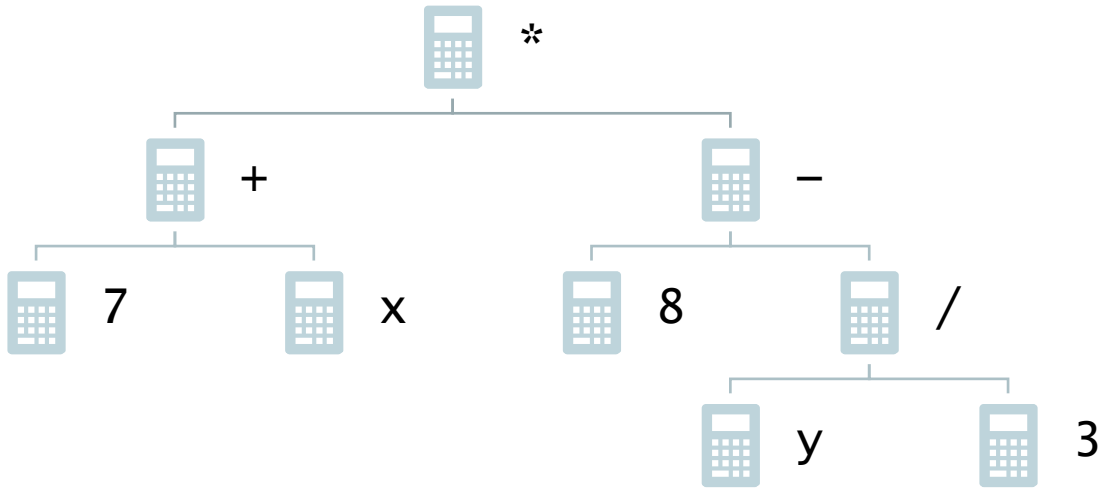


Programmeringssprogets semantiske regler fortæller, hvordan syntakstræet bruges til at beregne den værdi, udtrykket beskriver

Vi skal bare **oversætte** højniveau-koden til lavniveau-kode – og så afvikle lavniveau-koden

Semantik af udtryk

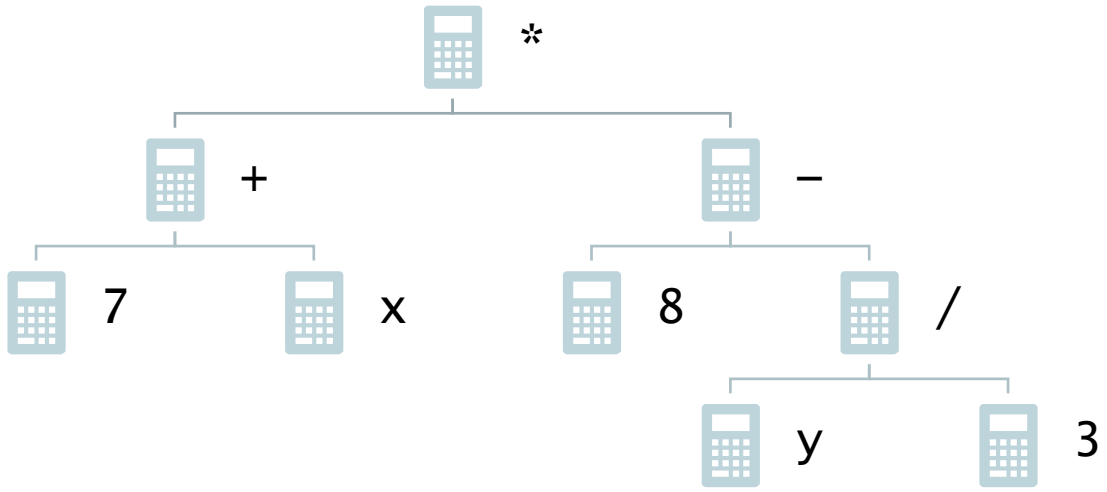
```
(7 + x) * (8 - y / 3)
```




```
code((7 + x) * (8 - y / 3))
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```



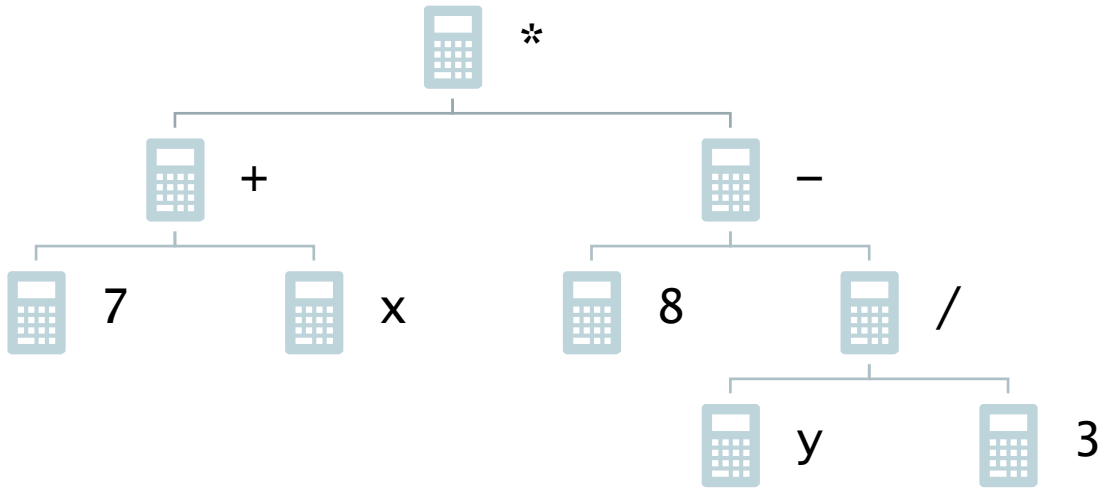

```
code((7 + x) * (8 - y / 3))

= ?
```

```
code(e1 * e2) = code(e1)
                code(e2)
                mul
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```




```
code((7 + x) * (8 - y / 3))

= code(7 + x)


  code(8 - y / 3)

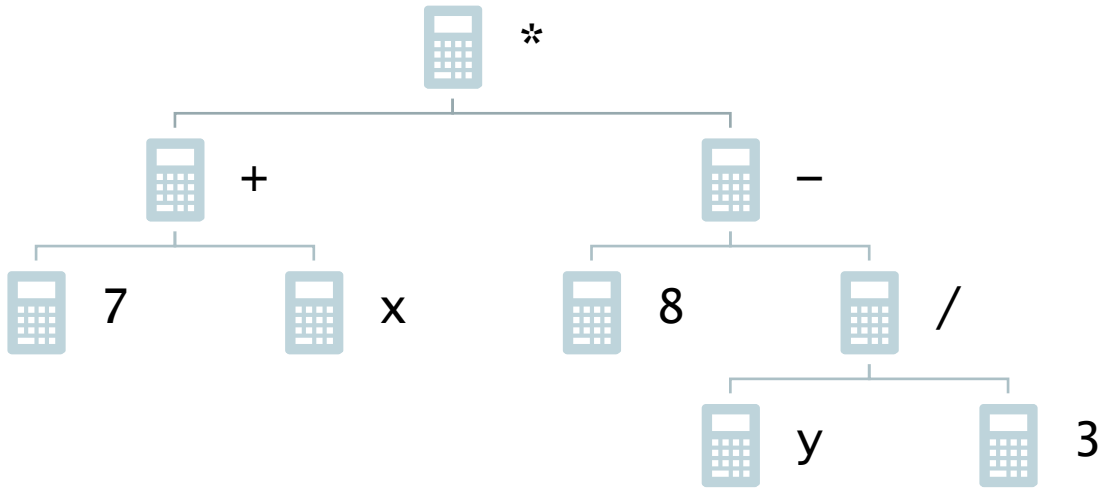



  mul
```

```
code(e1 * e2) = code(e1)
                code(e2)
                mul
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```




```
code((7 + x) * (8 - y / 3))

= code(7)
  code(x)
  add
  code(8 - y / 3)

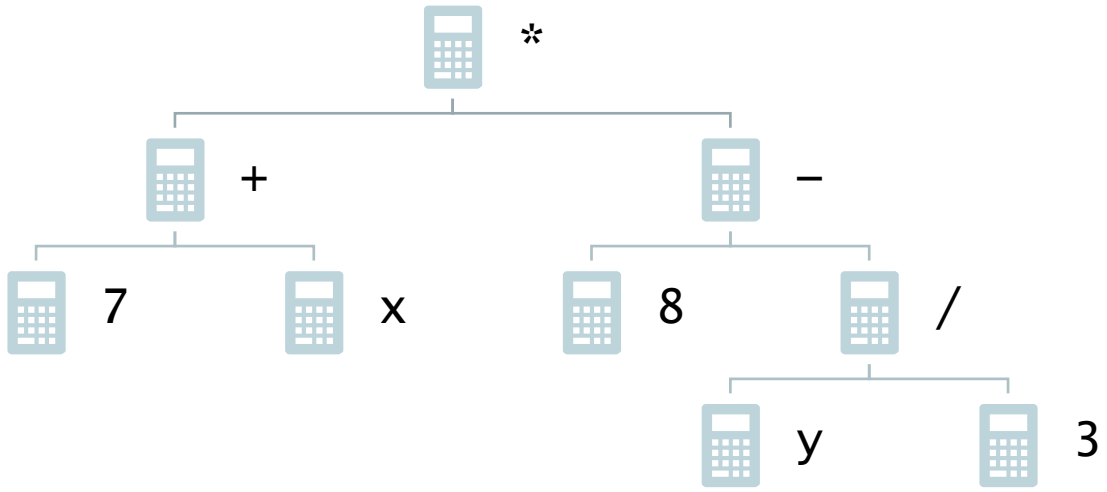


  mul
```

```
code(e1 + e2) = code(e1)
                code(e2)
                add
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```




```
code((7 + x) * (8 - y / 3))

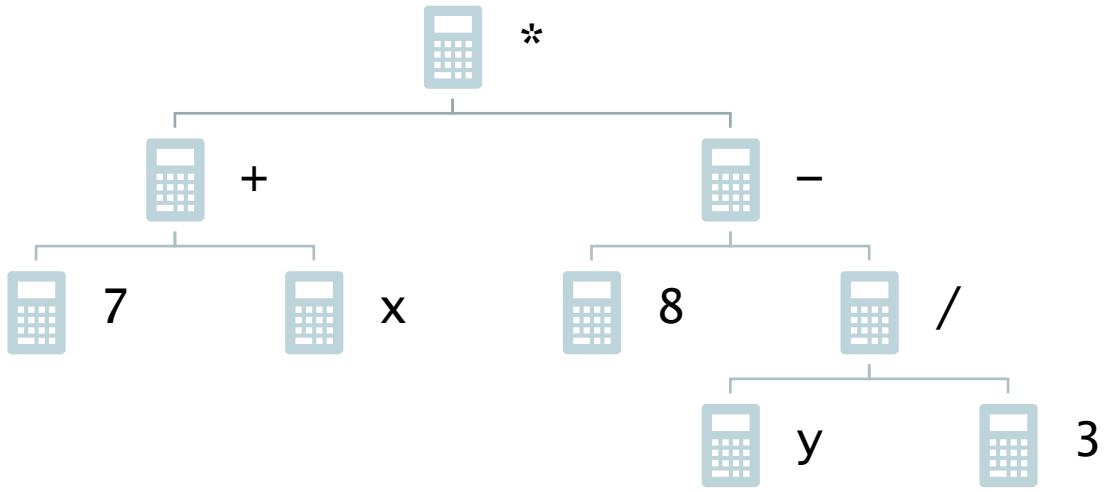
= code(7)
  code(x)
  add
  code(8)
  code(y / 3)


  sub
  mul
```

```
code(e1 - e2) = code(e1)
                code(e2)
                sub
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```



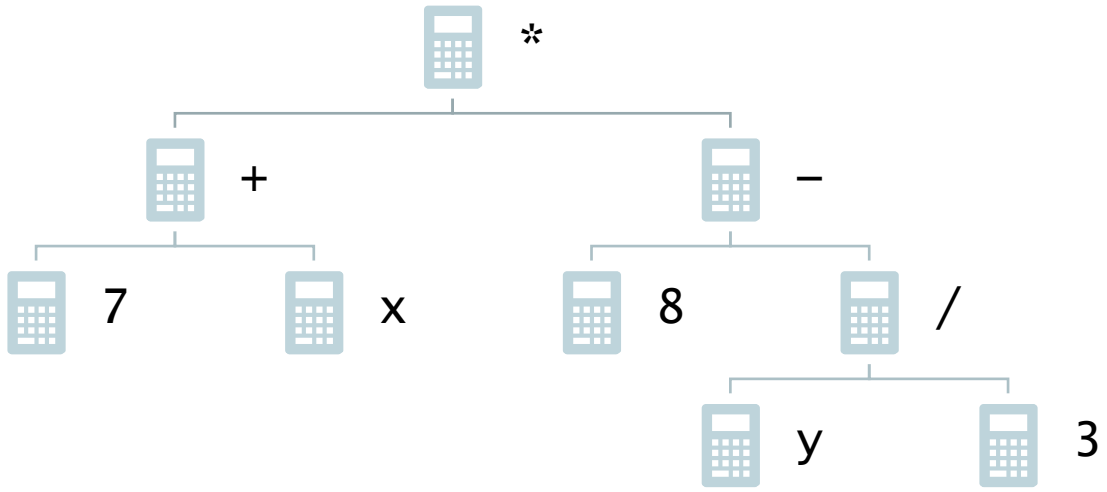

```
code((7 + x) * (8 - y / 3))

= code(7)
  code(x)
  add
  code(8)
  code(y)
  code(3)
  div
  sub
  mul
```

```
code(e1 / e2) = code(e1)
                code(e2)
                div
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```




```
code((7 + x) * (8 - y / 3))

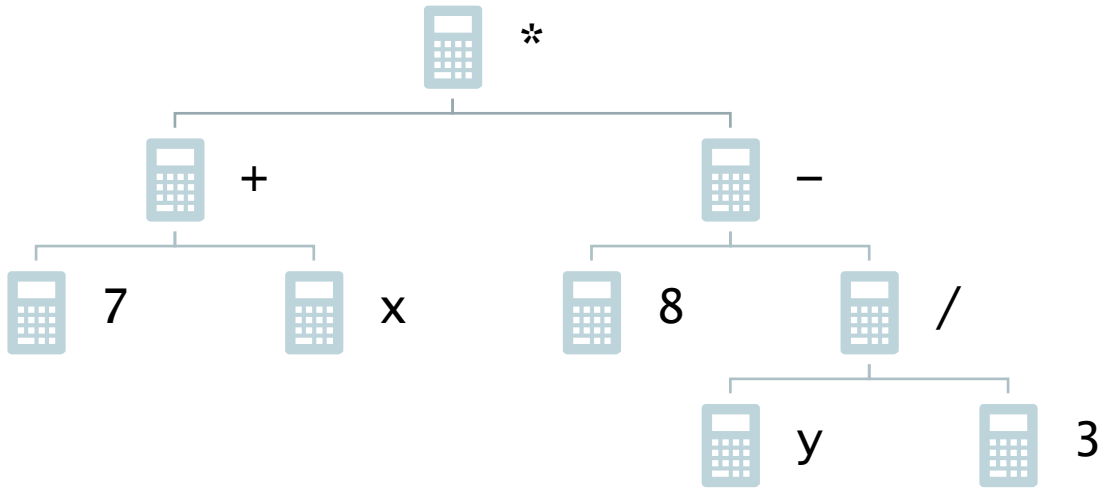
= push 7
  code(x)
  add
  push 8
  code(y)
  push 3
  div
  sub
  mul
```

code(n) = **push** n

– hvor n er et heltal

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```




```
code((7 + x) * (8 - y / 3))

= push 7
  load 0
  add
  push 8
  load 1
  push 3
  div
  sub
  mul
```

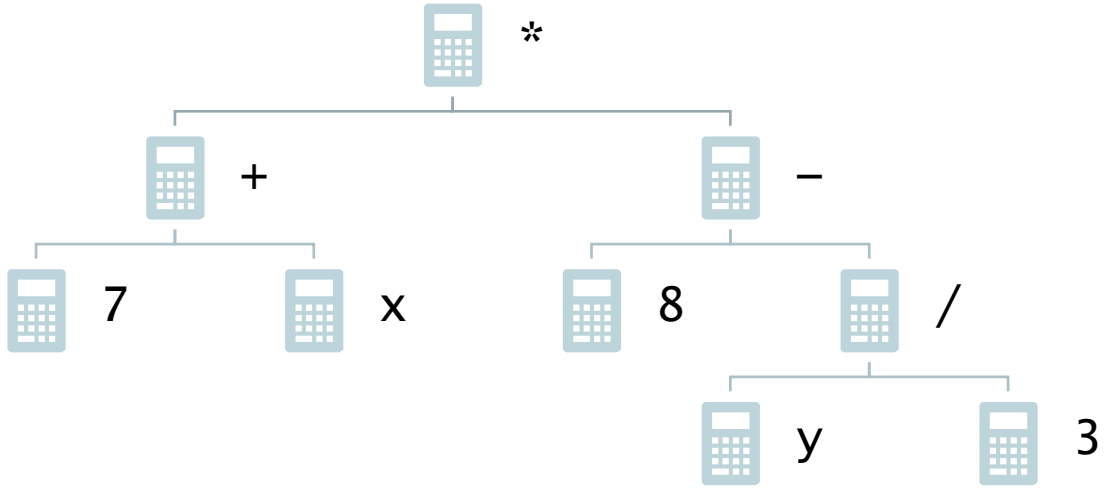
```
code(v) = load n
```

– hvis der er *n* variabler på stakken under variablen *v*

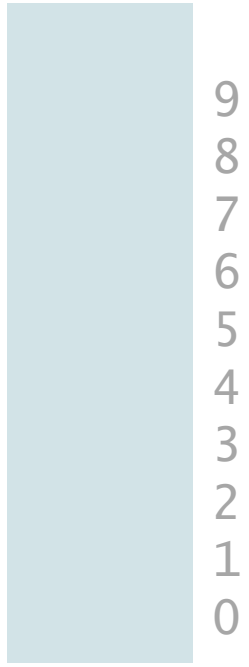
Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Klar til at afvikle lavniveau-koden?




```
push 7
load 0
add
push 8
load 1
push 3
div
sub
mul
```

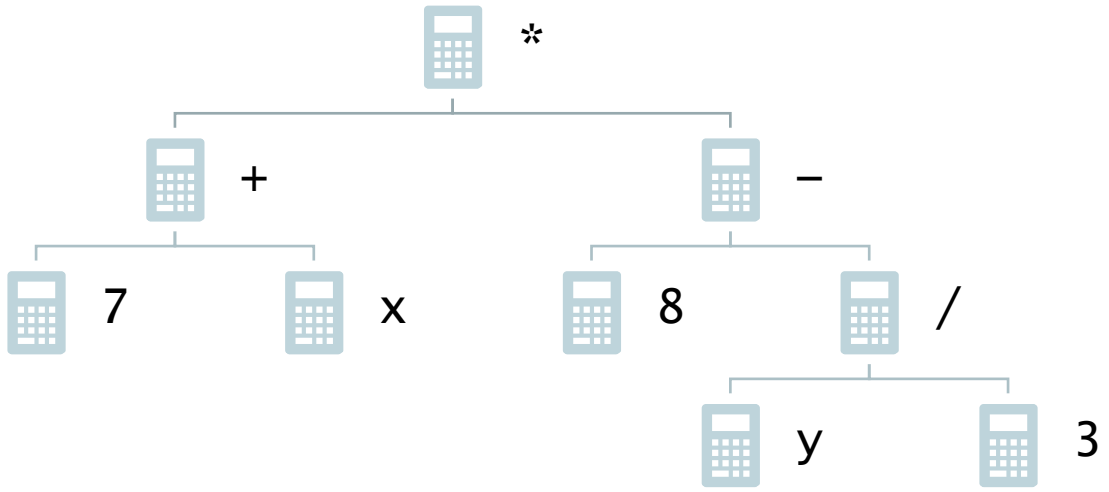


ERHVERVSAKADEMI
AARHUS

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15




```
> push 7
  load 0
  add
  push 8
  load 1
  push 3
  div
  sub
  mul
```

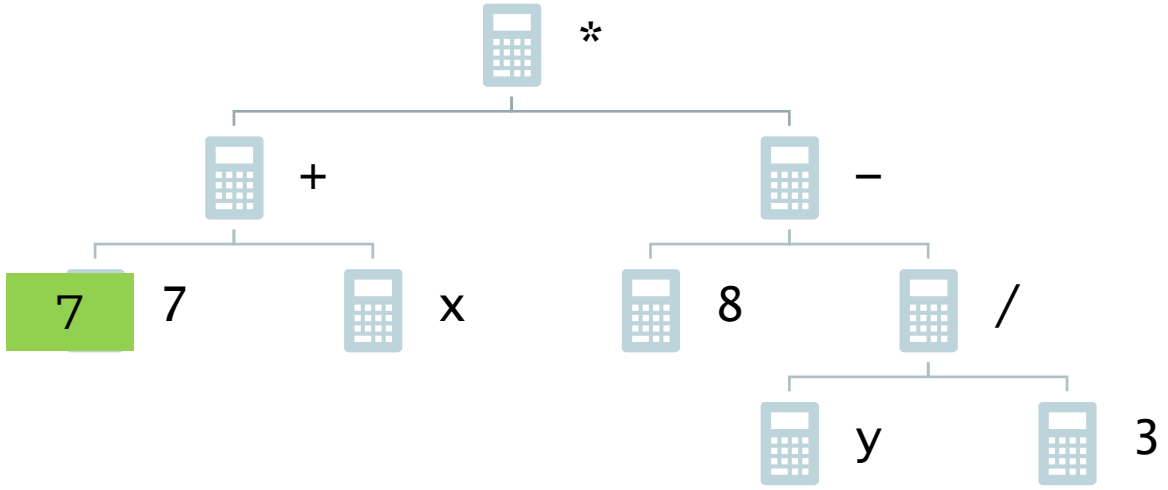
```
          9
          8
          7
          6
          5
          4
          3
          2
     15   1  y
      2   0  x
```

ERHVERVSAKADEMI
AARHUS

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15




```
   push 7
>  load 0
   add
   push 8
   load 1
   push 3
   div
   sub
   mul
```

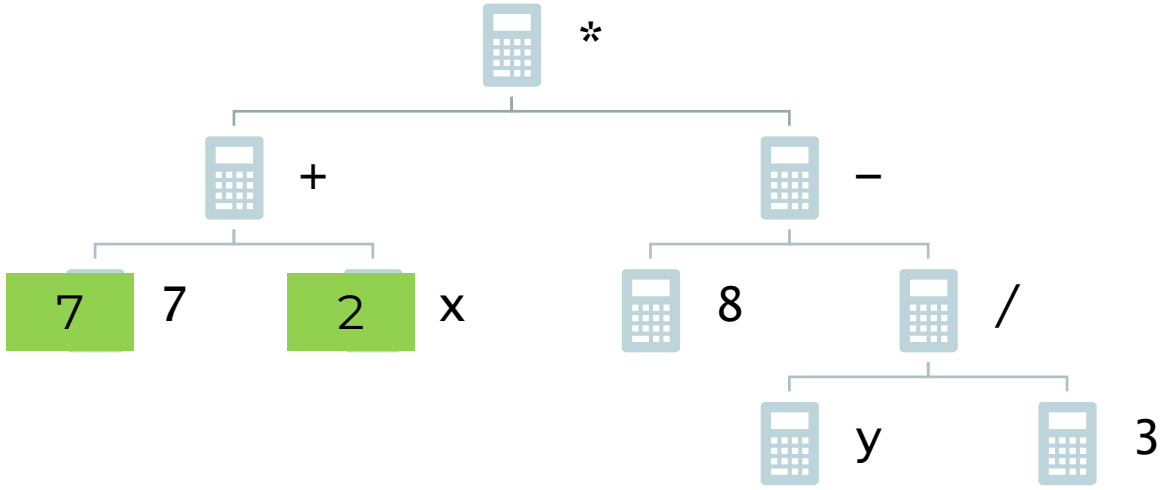
```
        9
        8
        7
        6
        5
        4
        3
        7  2
       15  1  y
        2  0  x
```

ERHVERVSAKADEMI
AARHUS

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15



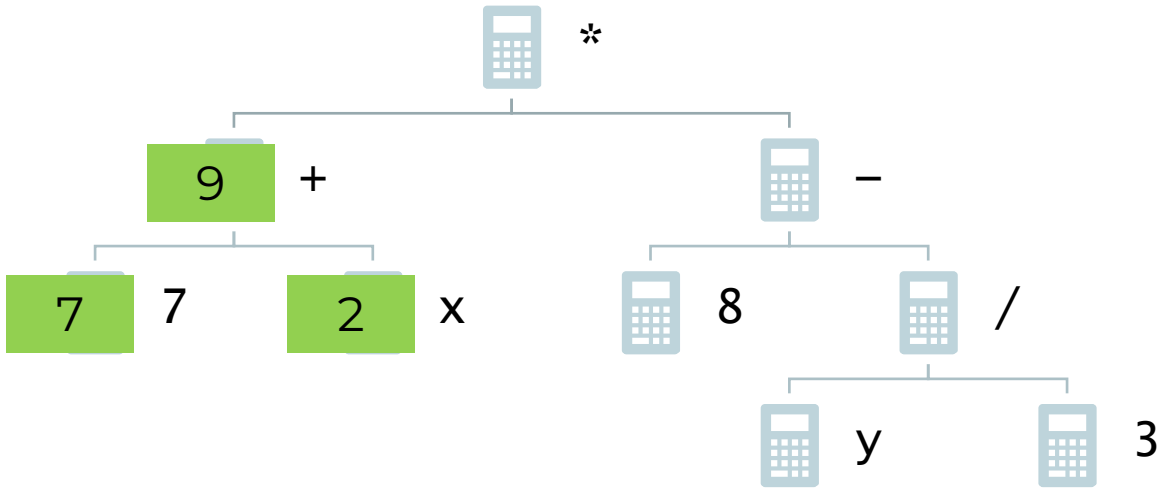

```
   push 7
   load 0
>  add
   push 8
   load 1
   push 3
   div
   sub
   mul
```

```
        9
        8
        7
        6
        5
        4
        3
        2
        1  y
        0  x
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15




```
   push 7
   load 0
   add
>  push 8
   load 1
   push 3
   div
   sub
   mul
```

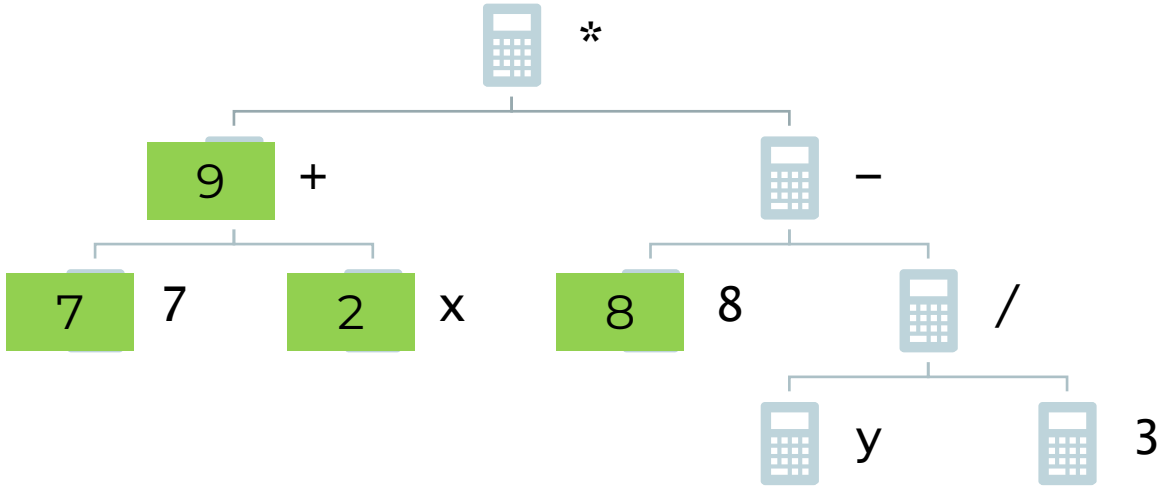
```
   9
   8
   7
   6
   5
   4
   3
   9   2
  15   1  y
   2   0  x
```

ERHVERVSAKADEMI
AARHUS

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15



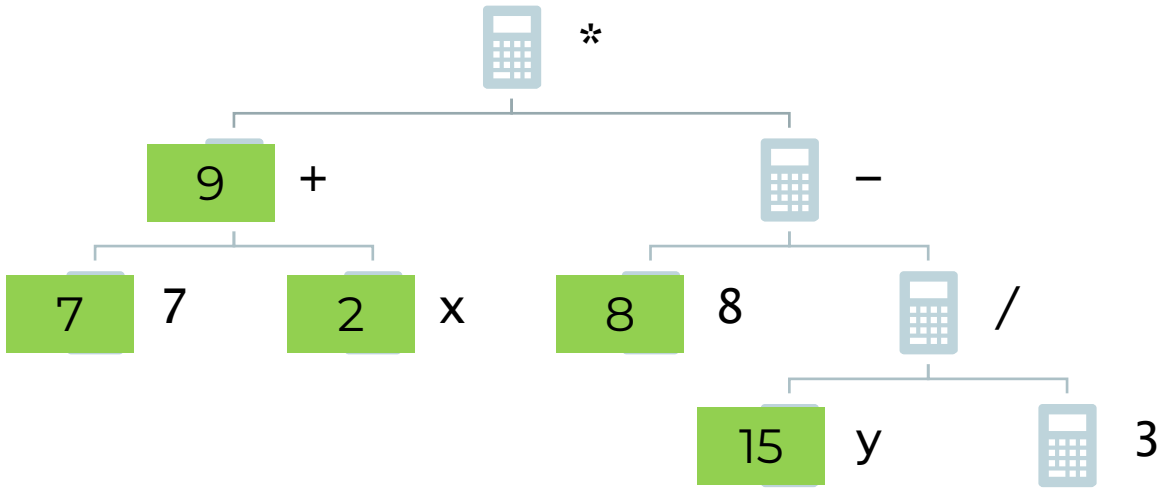

```
   push 7
   load 0
   add
   push 8
>  load 1
   push 3
   div
   sub
   mul
```

```
   9
   8
   7
   6
   5
   4
   8   3
   9   2
  15   1  y
   2   0  x
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15



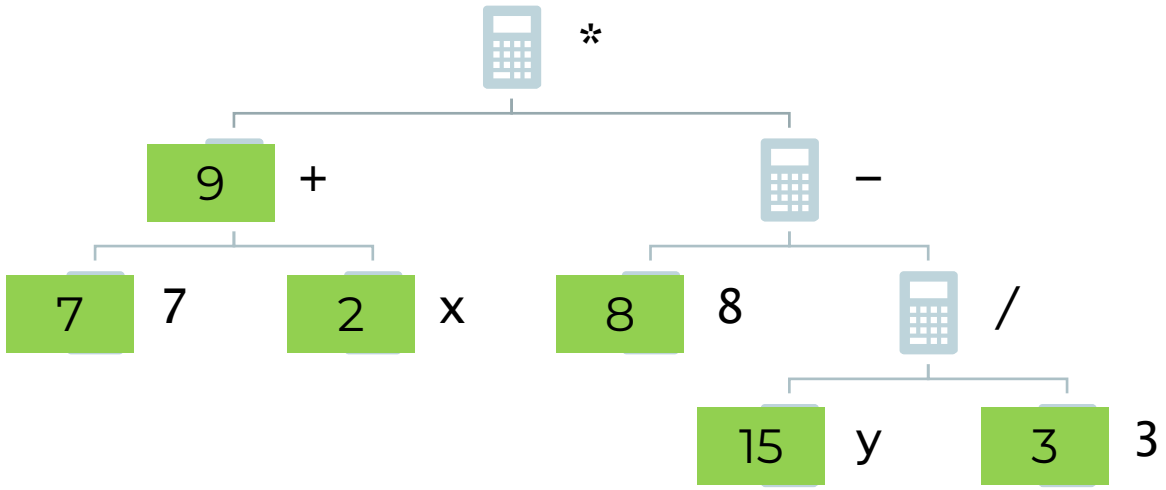

```
   push 7
   load 0
   add
   push 8
   load 1
>  push 3
   div
   sub
   mul
```

```
                9
                8
                7
                6
                5
       15       4
        8       3
        9       2
       15       1 y
        2       0 x
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15



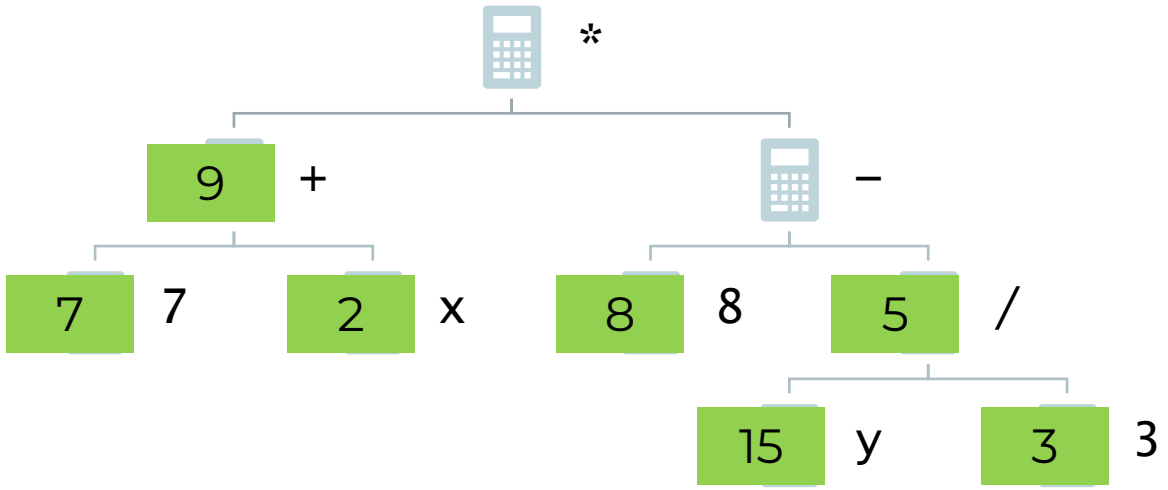

```
   push 7
   load 0
   add
   push 8
   load 1
   push 3
>  div
   sub
   mul
```

```
   9
   8
   7
   6
   3   5
  15   4
   8   3
   9   2
  15   1  y
   2   0  x
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15



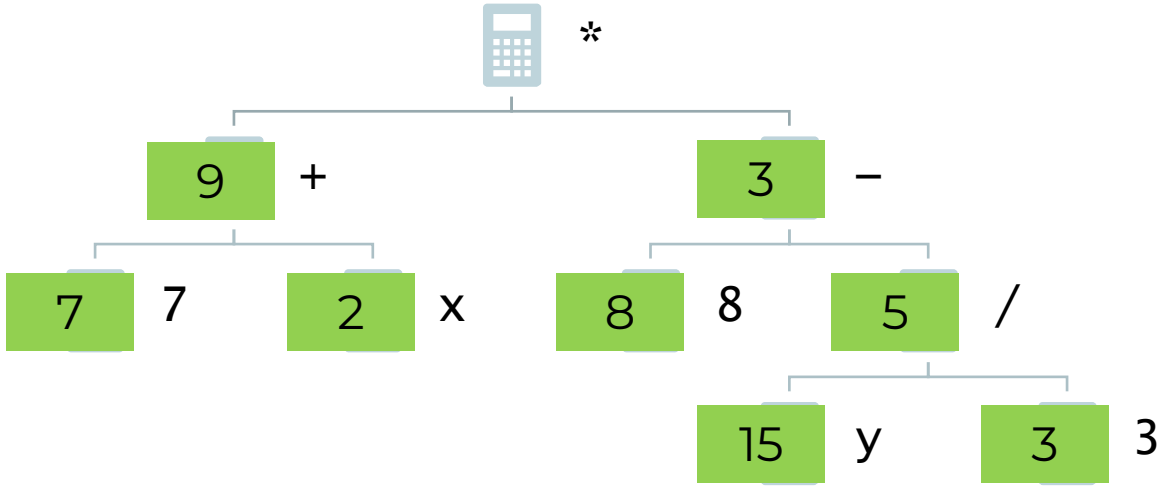

```
   push 7
   load 0
   add
   push 8
   load 1
   push 3
   div
>  sub
   mul
```

```
    9
    8
    7
    6
    5
    4  5
    3  8
    2  9
    1  15  y
    0  2   x
```

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15



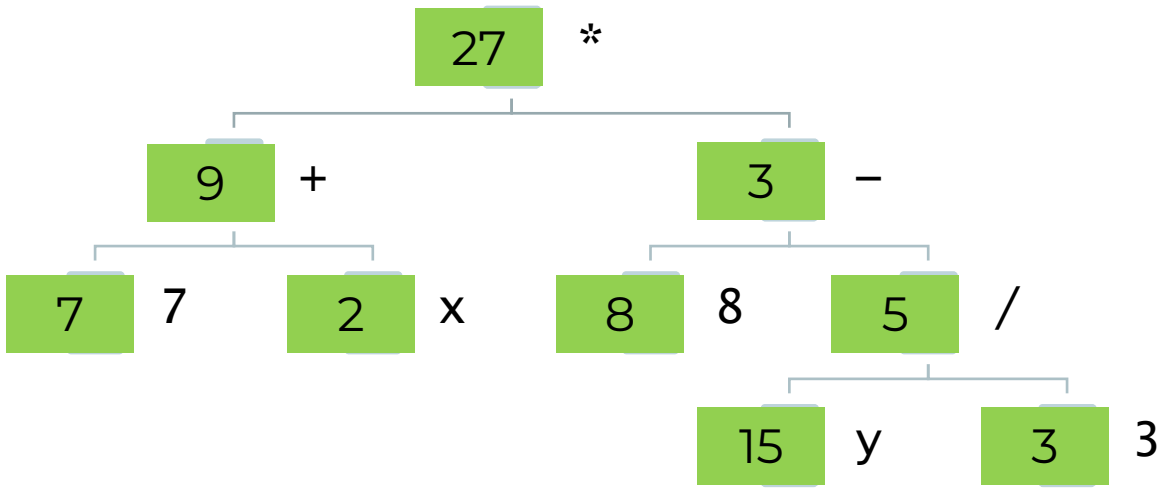

```
   push 7
   load 0
   add
   push 8
   load 1
   push 3
   div
   sub
> mul
```

		9
		8
		7
		6
		5
		4
	3	3
	9	2
	15	1 y
	2	0 x

Semantik af udtryk

```
(7 + x) * (8 - y / 3)
```

Lad os sige, at
x = 2 og y = 15




```
push 7
load 0
add
push 8
load 1
push 3
div
sub
mul
```

```
                9
                8
                7
                6
                5
                4
                3
       27       2
       15       1  y
        2       0  x
```

ERHVERVSAKADEMI
AARHUS

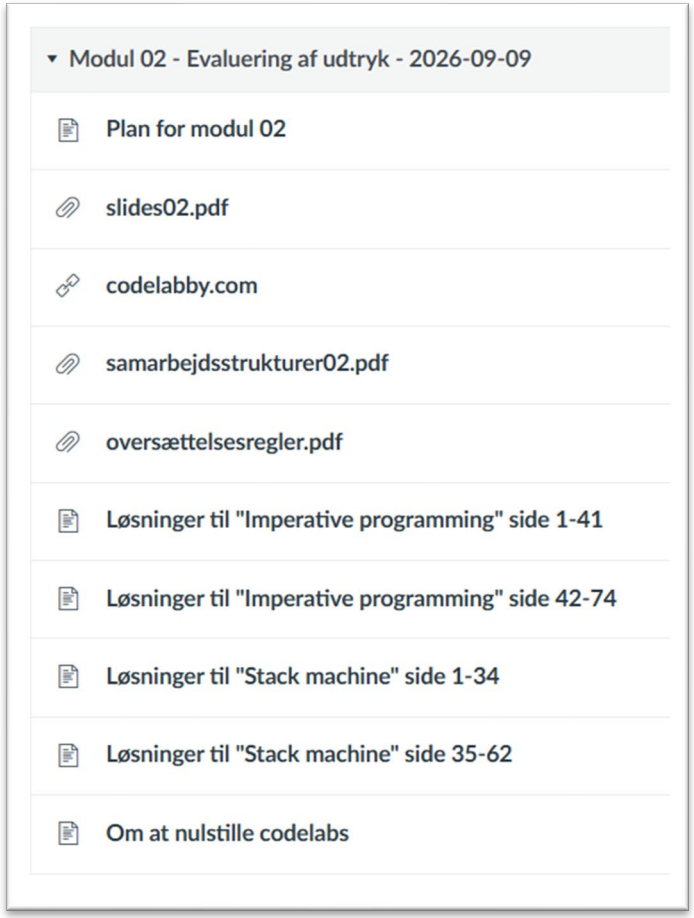
Codelabs

Følg slagplanen fra samarbejdsstrukturer02.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.
Pause 13:15–13:20

Ressourcer på Canvas:

- samarbejdsstrukturer og slagplan
- oversættelsesregler: højniveau til lavniveau
- opgaveløsninger med forklaringer
- om hvordan man nulstiller et codelab



ERHVERVSAKADEMI
AARHUS



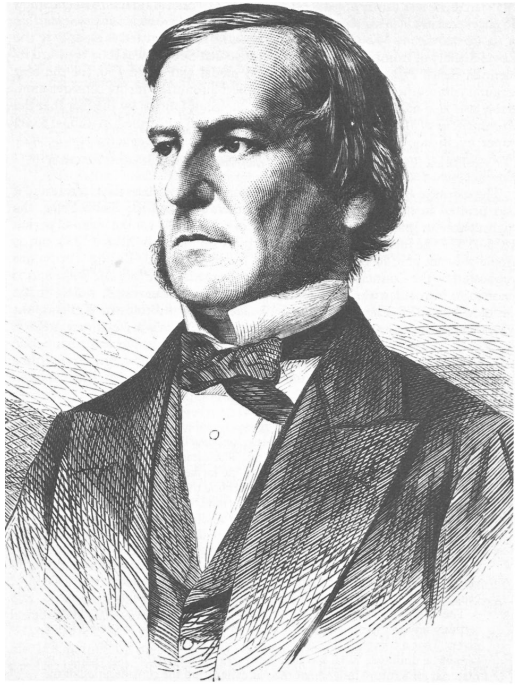
Sandhedsværdier / Booleans

Sandhedsværdi = boolesk værdi = Boolean sand eller falsk

Opkaldt efter George Boole, som omkring 1850 opdagede, at man kunne opstille en algebra for sandhedsværdier meget lig den, vi kender for tal:

or	false	true
false	false	true
true	true	true

and	false	true
false	false	false
true	false	true



Sandhedsværdier / Booleans

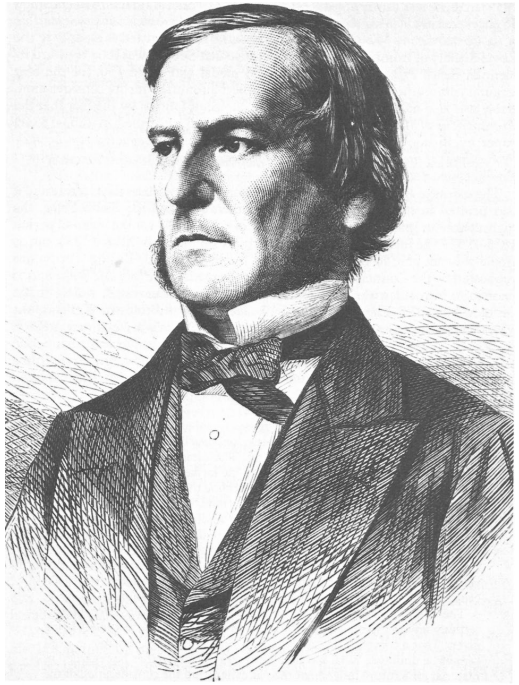
Sandhedsværdi = boolesk værdi = Boolean sand eller falsk

Opkaldt efter George Boole, som omkring 1850 opdagede, at man kunne opstille en algebra for sandhedsværdier meget lig den, vi kender for tal:

or	0	1
0	0	1
1	1	1

and	0	1
0	0	0
1	0	1

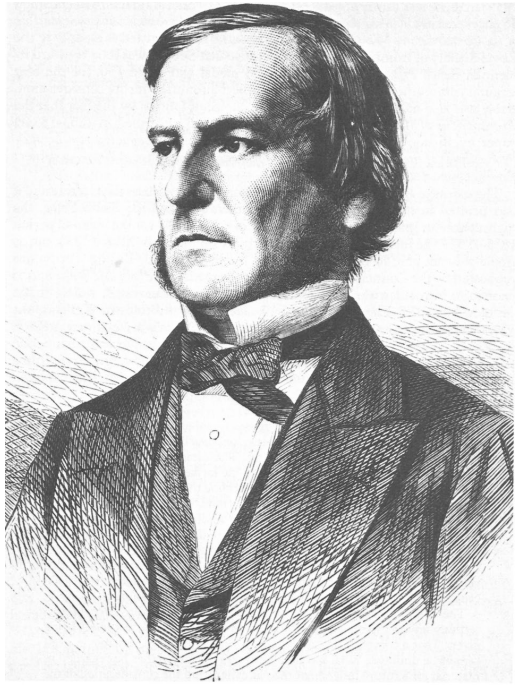
~ additions og multiplikationstabeller



Sandhedsværdier / Booleans

Sandhedsværdi = boolesk værdi = Boolean sand eller falsk

Opkaldt efter George Boole, som omkring 1850 opdagede, at man kunne opstille en algebra for sandhedsværdier meget lig den, vi kender for tal:



$x + 0 = x$	for alle tal x
b or false = b	for alle Booleans b
$x \times 1 = x$	for alle tal x
b and true = b	for alle Booleans b
$x \times (y + z) = (x \times y) + (x \times z)$	for alle tal x, y, z
a and (b or c) = (a and b) or (a and c)	for alle Booleans a, b, c

Sandhedsværdier / Booleans

Programmører kombinerer sandhedsværdier med and, or og not
med samme selvfølgelighed som matematikere kombinerer tal med +, −, × og ÷

Programmører bruger først og fremmest sandhedsværdier til at kontrollere kommandoer

```
while e do
  s
end while
```

```
if e then
  s
end if
```

```
if e then
  s1
else
  s2
end if
```

Sandhedsværdier / Booleans

Programmører kombinerer sandhedsværdier med and, or og not
med samme selvfølgelighed som matematikere kombinerer tal med +, −, × og ÷

Programmører bruger først og fremmest sandhedsværdier til at kontrollere kommandoer

Udtrykket **e** nedenfor skal evaluere til en sandhedsværdi:

```
while e do
   s
end while
```

```
if e then
   s
end if
```

```
if e then
   s1
else
   s2
end if
```

ERHVERVSAKADEMI
AARHUS

Sandhedsværdier / Booleans

Vi har set, hvordan man kan regne med tal i aritmetiske udtryk.

Hvordan beregner man en sandhedsværdi?

Sandhedsværdier / Booleans

Vi har set, hvordan man kan regne med tal i aritmetiske udtryk.

Hvordan beregner man en sandhedsværdi?

- ved at bruge booleske literaler
- ved at sammenligne tal
- ved at kombinere sandhedsværdier med andre sandhedsværdier ved hjælp af and, or og not

```
Both true and false are literal expressions.
```

```
If e1, e2, and e are expressions, then so are

 e1 = e2        e1 and e2
 e1 < e2        e1 or e2
 e1 > e2        not e
 e1 <> e2
 e1 <= e2
 e1 >= e2

The <> operator means "not equal to".
```

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer02.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Pause 14:05–14:25

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Evaluering af logiske udtryk

Evaluering af logiske udtryk

```
code(e1 and e2) = code(e1)
                  code(e2)
                  and

code(e1 or e2)  = code(e1)
                  code(e2)
                  or
```

Evaluering af logiske udtryk

```
code(e1 and e2) = code(e1)
                  code(e2)
                  and

code(e1 or e2)  = code(e1)
                  code(e2)
                  or
```

b1	b2	b1 and b2	b1 or b2
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Evaluering af logiske udtryk

```
code(e1 and e2) = code(e1)
                  code(e2)
                  and

code(e1 or e2)  = code(e1)
                  code(e2)
                  or
```

b1	b2	b1 and b2	b1 or b2
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

"Kortsluttet" evaluering

`e1 and e2`

Evaluér `e1`. Hvis værdien er `false`, så er det værdien af hele udtrykket.
Ellers evaluér `e2` og lad det være værdien af hele udtrykket.

`e1 or e2`

Evaluér `e1`. Hvis værdien er `true`, så er det værdien af hele udtrykket.
Ellers evaluér `e2` og lad det være værdien af hele udtrykket.

Evaluering af logiske udtryk

```
code(e1 and e2) = code(e1)
                  code(e2)
                  and

code(e1 or e2)  = code(e1)
                  code(e2)
                  or
```

b1	b2	b1 and b2	b1 or b2
tr			true
			true
fa			true
false		false	false

Forkert!
Med den oversættelse
får vi altid evalueret e2.

`e1 and e2`

Evaluér `e1`. Hvis værdien er `false`, så er det værdien af hele udtrykket.
Ellers evaluér `e2` og lad det være værdien af hele udtrykket.

`e1 or e2`

Evaluér `e1`. Hvis værdien er `true`, så er det værdien af hele udtrykket.
Ellers evaluér `e2` og lad det være værdien af hele udtrykket.

ERHVERVSAKADEMI
AARHUS

Evaluering af logiske udtryk

```
code(e1 and e2) = code(e1)
                  code(e2)
                  and

code(e1 or e2)  = code(e1)
                  code(e2)
                  or
```

b1	b2	b1 and b2	b1 or b2
tr			true
			true
fa			true
false		false	false

Forkert!
Med den oversættelse
får vi altid evalueret e2.

Vi er nødt til at kunne
springe over code(e2)
afhængig af resultatet
af at afvikle code(e1)

`e1 and e2`

Evaluér **e1**. Hvis værdien er **false**, så er det værdien af hele udtrykket.
Ellers evaluér **e2** og lad det være værdien af hele udtrykket.

`e1 or e2`

Evaluér **e1**. Hvis værdien er **true**, så er det værdien af hele udtrykket.
Ellers evaluér **e2** og lad det være værdien af hele udtrykket.

Betingede udtryk / conditionals

`e1 and e2`

Evaluér `e1`. Hvis værdien er `false`, så er det værdien af hele udtrykket.
Ellers evaluér `e2` og lad det være værdien af hele udtrykket.

`e1 or e2`

Evaluér `e1`. Hvis værdien er `true`, så er det værdien af hele udtrykket.
Ellers evaluér `e2` og lad det være værdien af hele udtrykket.

Betingede udtryk / conditionals

`e1 and e2`

Evaluér `e1`.
Hvis værdien er `true`, så evaluér `e2` og lad det være værdien af hele udtrykket.
Hvis værdien er `false`, så er det værdien af hele udtrykket.

`e1 or e2`

Evaluér `e1`.
Hvis værdien er `true`, så er det værdien af hele udtrykket.
Hvis værdien er `false`, så evaluér `e2` og lad det være værdien af hele udtrykket.

Betingede udtryk / conditionals

`e1 and e2`

Evaluér `e1`.
Hvis værdien er `true`, så evaluér `e2` og lad det være værdien af hele udtrykket.
Hvis værdien er `false`, så er det værdien af hele udtrykket.

`e1 or e2`

Evaluér `e1`.
Hvis værdien er `true`, så er det værdien af hele udtrykket.
Hvis værdien er `false`, så evaluér `e2` og lad det være værdien af hele udtrykket.

`e1 then e2 else e3`

Evaluér `e1`.
Hvis værdien er `true`, så evaluér `e2` og lad det være værdien af hele udtrykket.
Hvis værdien er `false`, så evaluér `e3` og lad det være værdien af hele udtrykket.

Betingede udtryk / conditionals

`e1 and e2`

Evaluér `e1`.
Hvis værdien er `true`, så evaluér `e2` og lad det være værdien af hele udtrykket.
Hvis værdien er `false`, så er det værdien af hele udtrykket.

`e1 or e2`

Evaluér `e1`.
Hvis værdien er `true`, så er det værdien af hele udtrykket.
Hvis værdien er `false`, så evaluér `e2` og lad det være værdien af hele udtrykket.

`e1 then e2 else e3`

Evaluér `e1`.
Hvis værdien er `true`, så evaluér `e2` og lad det være værdien af hele udtrykket.
Hvis værdien er `false`, så evaluér `e3` og lad det være værdien af hele udtrykket.

`e1 and e2`	betyder det samme som	`e1 then e2 else false`
`e1 or e2`	betyder det samme som	`e1 then true else e2`

Betingede udtryk / conditionals

```
code(e1 then e2 else e3) =
```

`e1` **`then`** `e2` **`else`** `e3`

Evaluér **`e1`**.
Hvis værdien er **`true`**, så evaluér **`e2`** og lad det være værdien af hele udtrykket.
Hvis værdien er **`false`**, så evaluér **`e3`** og lad det være værdien af hele udtrykket.

Betingede udtryk / conditionals

```
code(e1 then e2 else e3) = code(e1)
                           jmpf else
                           code(e2)
                           jump done
                           else:
                           code(e3)
                           done:
```

`e1` **`then`** `e2` **`else`** `e3`

Evaluér **`e1`**.
Hvis værdien er **`true`**, så evaluér **`e2`** og lad det være værdien af hele udtrykket.
Hvis værdien er **`false`**, så evaluér **`e3`** og lad det være værdien af hele udtrykket.

Betingede udtryk / conditionals

```
code(e1 then e2 else e3) = code(e1)
                           jmpf else
                           code(e2)
                           jump done
                           else:
                           code(e3)
                           done:

code(e1 and e2)          =
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3) = code(e1)
                           jmpf else
                           code(e2)
                           jump done
                           else:
                           code(e3)
                           done:

code(e1 and e2)          = code(e1 then e2 else false)
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3) = code(e1)
                           jmpf else
                           code(e2)
                           jump done
                           else:
                           code(e3)
                           done:

code(e1 and e2)          = code(e1 then e2 else false)
                         = code(e1)
                           jmpf else
                           code(e2)
                           jump done
                           else:
                           push false
                           done:
```

Betingede udtryk / conditionals

```
code(e1 then e2 else e3) = code(e1)
                           jmpf else
                           code(e2)
                           jump done
                           else:
                           code(e3)
                           done:

code(e1 or e2)           = code(e1 then true else e2)
                         = code(e1)
                           jmpf else
                           push true
                           jump done
                           else:
                           code(e2)
                           done:
```

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer02.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Pause 15:10–15:15

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Sideeffekter

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

```
given 3 4 then ?

write 7 + 3
write 7 * read
```

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

```
given 3 4 then 10 21

write 7 + 3
write 7 * read
```

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

```
given 3 4 then 10 21

write 7 + 3
write 7 * read
```

```
given 3 4 then ?

write 7 + read
write 7 * read
```

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

```
given 3 4 then 10 21

write 7 + 3
write 7 * read
```

```
given 3 4 then 10 28

write 7 + read
write 7 * read
```

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

```
given 3 4 then 10 21

write 7 + 3
write 7 * read
```

```
given 3 4 then 10 28

write 7 + read
write 7 * read
```

```
given 3 4 5 then ?

write read * (read + read)
```

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

```
given 3 4 then 10 21

write 7 + 3
write 7 * read
```

```
given 3 4 then 10 28

write 7 + read
write 7 * read
```

```
given 3 4 5 then 27

write read * (read + read)
```

Sideeffekter

Sideeffekt = noget, der sker under evaluering af et udtryk – ud over at værdien beregnes

Eksempler: læsning af input, skrivning af output, ændring af en variabel

```
given 3 4 then 10 21

write 7 + 3
write 7 * read
```

```
given 3 4 then 10 28

write 7 + read
write 7 * read
```

```
given 3 4 5 then 27

write read * (read + read)
```

Sideeffekter

- komplicerer tingene: samme udtryk betyder ikke nødvendigvis samme værdi
- tvinger os til at være eksplicitte omkring evaluerings-rækkefølgen

Sideeffekter

Nogen gange giver det mening at evaluere et udtryk blot for at få sideeffekterne

```
given 3 4 expect 28

// skip first input?
write 7 * read
```

Sideeffekter

Nogen gange giver det mening at evaluere et udtryk blot for at få sideeffekterne

```
given 3 4 expect 28

read // skip first input?
write 7 * read
```

Sideeffekter

Nogen gange giver det mening at evaluere et udtryk blot for at få sideeffekterne

```
given 3 4 expect 28

read // skip first input?
write 7 * read
```

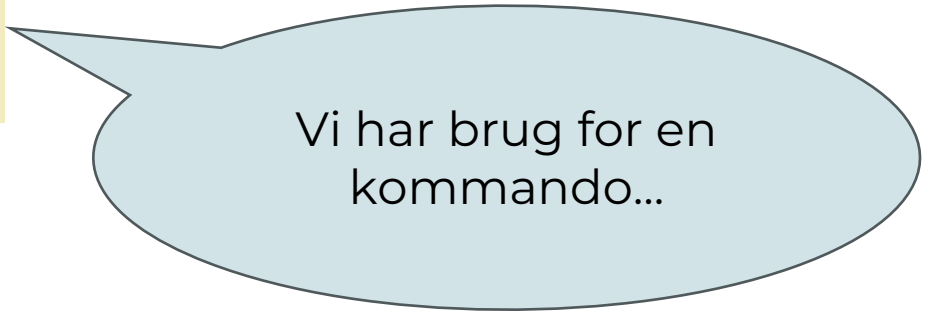


Sideeffekter

Nogen gange giver det mening at evaluere et udtryk blot for at få sideeffekterne

```
given 3 4 expect 28

eval read // skip first input
write 7 * read
```



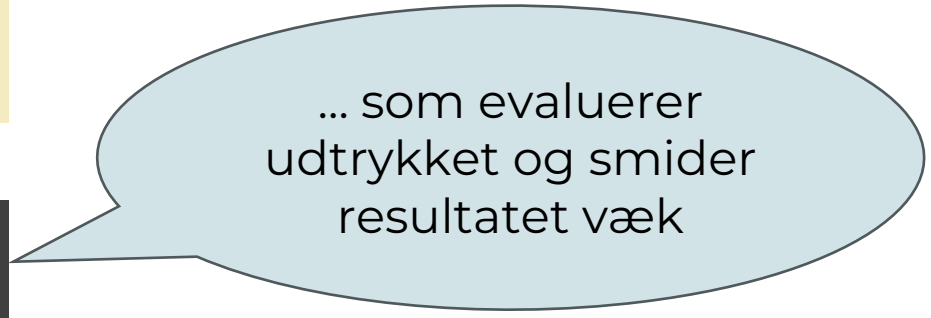
Sideeffekter

Nogen gange giver det mening at evaluere et udtryk blot for at få sideeffekterne

```
given 3 4 expect 28

eval read // skip first input
write 7 * read
```

```
code(eval e) = code(e)
               pop
```



ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer02.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Opsamling 15:50

ERHVERVSAKADEMI
AARHUS

Opsamling

Relationelle og logiske udtryk

Betingede udtryk

Kortslutninger og sideeffekter

Hvad var svært at forstå?

ERHVERVSAKADEMI
AARHUS

Opfølgende hjemmearbejde

Gennemgå "Imperative programming" side 1-41 flere gange

Gennemgå "Stack machine" side 1-48 flere gange

Det er øvelsesbaner, ikke afleveringer

Nulstil codelabs
mellem gentagelserne

Bliv ved, indtil

- teksten føles velkendt og let forståelig
- du kan løse opgaverne uden hjælp

Brug din studiegruppe så meget som muligt

- brug de nye begreber mundtligt: spørg, forklar, eksemplificér
- vær konkret og specifik

ERHVERVSAKADEMI
AARHUS

Opfølgende hjemmearbejde

Gennemgå "Imperative programming" side 1-41 flere gange

Gennemgå "Stack machine" side 1-48 flere gange

Det er øvelsesbaner, ikke afleveringer

Hjernen skal have
tid og gentagelser
for at lære effektivt

Tommelfingerregel: 10/24/7

Bliv ved, indtil

- teksten føles velkendt og let forståelig
- du kan løse opgaverne uden hjælp

Brug din studiegruppe så meget som muligt

- brug de nye begreber mundtligt: spørg, forklar, eksemplificér
- vær konkret og specifik

ERHVERVSAKADEMI
AARHUS

Reaktioner på i dag

Spørgsmål?

Bekymringer?

Protester?

Kritik?